GODSMINO: Design Specification, MVVM Architecture, and Implementation of a Real-Time Domino Game

Miguel
Software Engineering — Mobile Application Project
`com.example.godsmino` v1.0

***Abstract*—GODSMINO is an Android application implementing a double-six domino game extended with five special powers and a random event system. It supports single-player matches against a greedy-strategy AI and real-time two-player online matches synchronized through Firebase Firestore. The project follows the MVVM architectural pattern with Jetpack Compose for the declarative UI and Kotlin Coroutines/StateFlow for reactive state management. This report documents the design specifications, MVVM layer structure, UML class diagram, component catalog, scoring system, special powers, random events, and all application screens.**

***Index Terms*—Android, Jetpack Compose, MVVM, Firebase Firestore, Kotlin, Domino, StateFlow**

I. Introduction

GODSMINO is a real-time multiplayer domino game for Android. The application combines the rules of double-six domino with modern game mechanics: **special powers** that grant limited strategic advantages, and **random events** that temporarily modify game conditions during a match.

The technical stack is: **Jetpack Compose** (declarative UI), **MVVM** architectural pattern (separation of concerns), **Firebase Firestore** (real-time state synchronization), and **Firebase Auth** (user identity). Two game modes are supported:

- **Quick Match vs Bot:** Single-player against a greedy AI.
- **Online Room:** Real-time two-player via room codes.

II. Design Specifications

A. Initial Configuration

The game uses a standard double-six set of 28 tiles (from `[0|0]` to `[6|6]`). All tiles are generated via a nested loop ($i \in [0, 6]$, $j \in [i, 6]$), shuffled, and distributed as follows:

Table I
Tile distribution by game mode

Mode	Tiles per player	Boneyard
Bot (offline)	7 each	14 tiles
Online	7 to host; 7 from boneyard on join	14 (after join)

B. Turn Dynamics and Move Validation

The current turn is tracked by `currentTurnPlayerId`. On their turn, a player selects a tile and the system validates it against the open board ends (`getLeftEnd()`, `getRightEnd()`). If the tile matches a board end, it is placed and aligned automatically (flipped if required so the connecting value faces inward). If no valid tile exists, the player draws from the boneyard; drawing does not pass the turn. If the boneyard is empty and no move is possible, the player passes.

C. Game States

Table II
Game state enumeration

State	Description
`WAITING`	Room created, waiting for opponent.
`COIN_TOSS`	Coin-toss animation running.
`PLAYING`	Match in progress.
`FINISHED`	Match ended.

Figure 1 shows the transition diagram between states.

D. Coin-Toss Mechanism

Before each online match, the host calls `startCoinToss()`, which selects a random winner, sets the status to `COIN_TOSS`, and stores the winner's player ID in `activeEvent.effectType`. The `CoinTossDialog` displays a spinning animated coin for 3.2 seconds. The visual result depends on the winner:

- **Player 1 wins:** The coin shows "Heads!" (crown symbol).
- **Player 2 wins:** The coin shows "Tails!" (sword symbol).

After a 2-second reveal phase, the host calls `finishCoinToss(winnerId)`, which sets `status = PLAYING` and assigns `currentTurnPlayerId` to the winner. The dialog reads the winner directly from `session.activeEvent.effectType`, making the result deterministic and synchronized across devices.

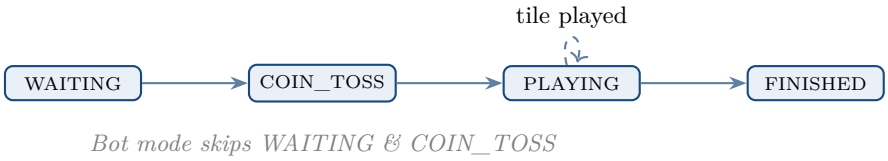


Figure 1. Game state machine. Bot mode starts directly in PLAYING.

III. MVVM Architecture

GODSMINO strictly implements the **Model-View-ViewModel** pattern to ensure separation of concerns, UI reactivity, and testability.

A. Layer Diagram

B. View Layer

Compose composables observe `StateFlow<GameUiState>` via `collectAsState()` and send user intents to the ViewModel via function calls. No business logic or Firebase code lives in the View layer.

C. ViewModel Layer

`GameViewModel` (extends `AndroidViewModel`) exposes `StateFlow<GameUiState>`. The `GameUiState` holds:

- `session: GameSession` — current game state.
- `selectedTile: Tile?` — tile selected by the player.
- `selectedPower: PowerType?` — active power selection.
- `pendingPowerDialog: PowerType?` — power awaiting confirmation.
- `selectedTileForSwap: Tile?` — tile chosen for SWAP.
- `boardThemeIndex: Int` — board theme (0–3).
- `errorMessage: String?` — error to display.

Additional responsibilities: auto-join for `p2`, statistics recording on `FINISHED`, bot AI dispatch after 1200 ms, and an anti-regression guard (ignores remote `WAITING` snapshots when already `PLAYING`).

D. Repository Layer

`GameRepository` is the game engine and single source of truth. It owns all rule logic (tile dealing, move validation, scoring, blocked-game detection, powers, random events, bot). It holds a `MutableStateFlow<GameSession>` in memory and calls `syncGameSession()` after every state mutation.

E. Reactive Data Flow

1) User interacts with the UI.
2) View calls a ViewModel function.
3) ViewModel calls the Repository.
4) Repository mutates `GameSession` and calls `syncGameSession()`.
5) DataSource updates the in-memory flow and writes to Firestore.
6) Repository flow emits; ViewModel updates `StateFlow`.
7) View recomposes automatically.

IV. Class Diagram

A. Data Models and Enumerations

Table III
Core data model classes

Class	Key fields and methods
`Tile`	`id: String` (UUID), `left: Int`, `right: Int`; `isDouble()`, `sumPoints()` (`@Exclude`)
`Player`	`id`, `name`, `score: Int`, `tiles: List<Tile>`, `powerCounts: Map<String,Int>`
`GameSession`	`roomId`, `board: List<Tile>`, `players: List<Player>`, `boneyard: List<Tile>`, `status: GameStatus`, `activeEvent: RandomEvent?`, `roundNumber: Int`; `boneyardSize()`, `getLeftEnd()`, `getRightEnd()`
`RandomEvent`	`id`, `title`, `description`, `effectType: String`
`User`	`uid`, `email`, `displayName`

Table IV
Enumerations

Enum	Values
`PowerType`	`WILDCARD`, `SKIP`, `SWAP`, `REVERSE`, `FORCE_DRAW_THREE`; each with `displayName`, `description`, `maxUses`, `icon`
`GameStatus`	`WAITING`, `COIN_TOSS`, `PLAYING`, `FINISHED`
`BoardDirection`	`LEFT`, `RIGHT`, `UP` (alias LEFT), `DOWN` (alias RIGHT)

B. Repository and ViewModel Classes

C. UML Relationships

Figure 3 summarizes the structural relationships of the main classes.

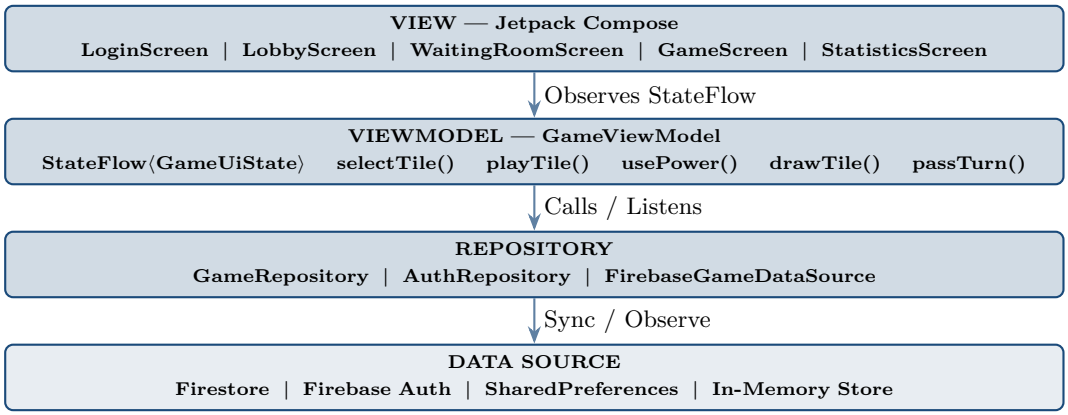


Figure 2. MVVM architecture layers of GODSMINO.

Table V
Repository and ViewModel classes

Class	Key responsibilities
`GameRepository`	Game engine: `makeMove()`, `drawTile()`, `passTurn()`, `usePower()`, `forfeitGame()`, `startCoinToss()`, `finishCoinToss()`, `evaluateTranca()`, `generateRandomEvent()`
`AuthRepository`	Authentication: `login()`, `register()`, `loginAsGuest()`, `signInWithCredential()`, `logout()`
`FirebaseGame-DataSource`	Firestore sync: `syncGameSession()`, `observeGameSession()`, in-memory store
`GameViewModel`	UI orchestration: exposes `StateFlow<GameUiState>`, drives bot AI, records stats

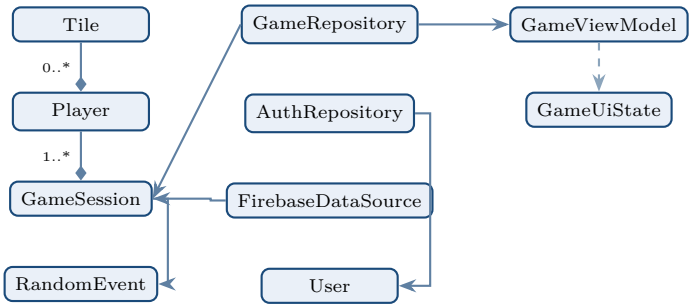


Figure 3. UML relationships. Diamond = composition; solid = association; dashed = exposes.

V. Architecture and Components

A. Application Entry Points

Two activities are declared in `AndroidManifest.xml`:

`GodsminoApp` (extends `Application`) initializes `FirebaseApp` in `onCreate()`, wrapped in `try/catch`. Declared permissions: `INTERNET`, `ACCESS_NETWORK_STATE`.

Table VI
Declared activities

Activity	Exp.	Role
`AuthActivity`	true	Launcher (MAIN/LAUNCHER). Shows login screen or redirects if a session exists.
`MainActivity`	false	Navigation host; manages screen state and `GameViewModel` instance.

B. Navigation

`MainActivity` manages a `when`-based screen state: `"lobby"`, `"waiting_room"`, `"game"`, `"stats"`. Figure 4 shows the navigation flow.

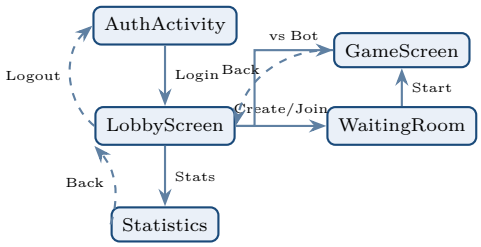


Figure 4. Navigation flow between screens.

C. Reusable UI Components

D. Utility Objects

GameStatsManager (singleton) persists match statistics in `SharedPreferences`: wins, losses, total matches, high score, and a JSON history of the last 20 matches. Called by the ViewModel on `FINISHED`.

SoundManager (singleton) synthesizes audio via `AudioTrack` (PCM 16-bit, 22050 Hz mono) without audio files:

VI. Scoring System

GODSMINO implements three match termination conditions. Points accumulate in `Player.score` across rounds.

Table VII
Reusable UI components

Component	Description
`DominoTileView`	Renders a tile: gradient fill, metallic border (thicker when selected), scale animation on selection, pip Canvas drawing. Horizontal 104×58 dp; doubles render vertically.
`GameBoardView`	Horizontally scrollable board; 4 selectable themes; floating open-end indicators. Uses `DominoTileView`.
`CoinTossDialog`	Full-screen overlay; spinning coin (3.2 s); winner from `activeEvent.effectType`; Heads (P1) / Tails (P2); 2 s reveal.
`PowerConfirmDialog`	Confirms power use; for SWAP shows player's hand for tile selection.
`RandomEventBanner`	Fade-in/out banner showing active event title and description.
`ConfettiOverlay`	Canvas: 120 colored circles on `FINISHED`.
`OpponentHeaderView`	Opponent avatar, name, thinking indicator, and score.

Table VIII
Synthesized sound effects

Method	Tone / Pattern
`playTileClack`	800 / 400 Hz
`playPowerSound`	C5–E5–G5 arpeggio
`playDrawTile`	300 Hz
`playVictorySound`	C5–E5–G5–C6
`playClickSound`	1200 Hz

A. Tile Point Value

$$\text{points}(t) = t.\texttt{left} + t.\texttt{right}$$

B. Condition 1 — Domino (Empty Hand)

When a player empties their hand:

$$\Delta\text{score}_{\text{winner}} = \sum_{\text{opponent tiles}} \text{points}(t) + 50$$

The match ends with `FINISHED`; `currentTurnPlayerId` stays on the winner.

C. Condition 2 — Blocked Game

Triggered in `evaluateTranca()`, called from `passTurn()`, when: the board is non-empty, the boneyard is empty, and neither player can place a tile.

1) Compute $S_i = \sum_{\text{tiles of player } i} \text{points}(t)$ for $i \in \{1, 2\}$.
2) If "Crazy Tile" event (`ev_loca`) is active, double the value of each double tile in S_i.
3) The player with the lower S_i wins.
4) Tiebreaker: fewer powers used ($\text{used}_i = 5 - \sum \texttt{powerCounts}_i$).
5) Absolute tie: player 1 wins.

Winner's score increment: $\sum_{\text{opponent tiles}} \text{points}(t) + 50$.

D. Condition 3 — Forfeit

$\Delta\text{score}_{\text{opponent}} = +100$.

Table IX
Termination conditions and scoring

Condition	Criterion	Winner gains
Domino	Player empties hand	Σ opp. tiles + 50
Blocked	Neither can play	Σ opp. tiles + 50
Forfeit	Player abandons the match	+100 fixed

VII. Special Powers

Each player starts with a fixed number of charges per power, initialized automatically from `PowerType.maxUses`:

```
val powerCounts: Map<String, Int> =
  PowerType.entries.associate { it.name to it.maxUses }
```

Listing 1. Power charge initialization.

A. Power Catalog

B. Activation Flow

Figure 5 shows the complete activation sequence.

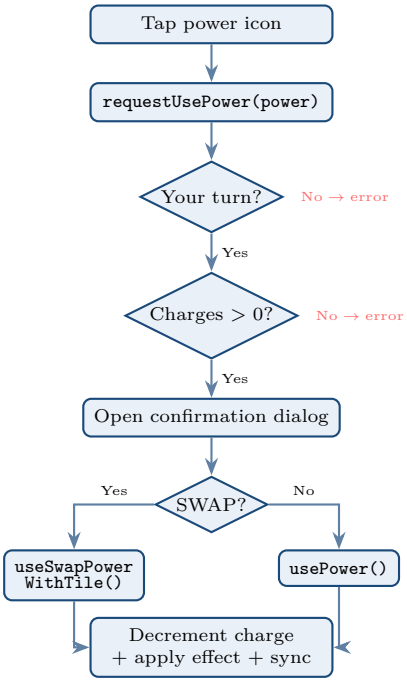


Figure 5. Power activation flow.

C. Charge Validation

```
val count = player.powerCounts[powerType.name] ?: 0
if (count <= 0)
    return Pair(false, "No charges remaining.")
val updated = player.powerCounts.toMutableMap()
updated[powerType.name] = count - 1
```

Listing 2. Charge check before use.

Table X
Special powers catalog

Enum	Name	Uses	Description	Implementation effect
WILDCARD	Wildcard	2	Places any tile without matching board ends.	Next makeMove() with useWildcard=true skips end validation. selectedPower stays WILDCARD until used.
SKIP	Skip	1	Skips the opponent's turn.	currentTurnPlayerId remains on the active player.
SWAP	Swap	1	Exchanges one of your tiles for a random opponent tile.	Player selects the tile to give; opponent's tile is chosen randomly.
REVERSE	Reverse	1	Reverses turn order.	In 2-player mode, equivalent to playing again (currentTurnPlayerId unchanged).
FORCE_DRAW_THREE	Force Draw	1	Forces opponent to draw up to 3 tiles from the boneyard.	Transfers min(3, |boneyard|) tiles to opponent's hand.

VIII. Random Events

After each non-finishing tile placement, generateRandomEvent() is triggered with 20% probability:

```
val event = if (!isFinished && Random.nextFloat() < 0.20f)
                generateRandomEvent()
            else current.activeEvent
```

Listing 3. Random event trigger.

Table XI
Random event catalog

ID	Title	Type	Effect
ev_pozo	Tile Storm	POZO	Informational only.
ev_loca	Crazy Tile	SCORE	**Coded:** doubles value of double tiles in blocked-game scoring.
ev_bonus	Blessing	SCORE	Informational only.
ev_inv	Inversion	TURN	Informational only.

Only ev_loca has a coded mechanical effect; the others are displayed in the RandomEventBanner for informational purposes.

IX. Application Features

A. Authentication

Three methods, all with offline fallback:

- **Email / Password:** createUserWithEmailAndPassword() and signInWithEmailAndPassword().
- **Guest:** Anonymous authentication via signInAnonymously().
- **Google:** AccountManager.newChooseAccountIntent() device picker; registers with email and fixed password.

If Firebase is unreachable, a local User is created with a UID derived from the email hash, ensuring full offline functionality.

B. Game Modes

Quick Match vs Bot: Room ID "room_offline_bot", both players seated immediately, status PLAYING, no network access, all sync in-memory.

Online Room: Host generates code "GDS" + random 100-999. Guest enters code in LobbyScreen. WaitingRoomScreen shows the code (with copy button), player slots, and a host-only "Start Match!" button (enabled with ≥ 2 players). Starting initiates the coin toss.

C. Bot AI

The bot (p2) applies a greedy strategy with a 1200 ms thinking delay:

1) Play the first valid tile, preferring the right board end.
2) If no valid tile, draw from boneyard.
3) If boneyard empty, try SKIP power.
4) If SKIP unavailable, pass turn.

D. Firebase Integration

FirebaseGameDataSource combines in-memory and Firestore storage:

- **In-memory store:** Map<String, MutableStateFlow<GameSession?>> for instant same-process propagation.
- **Firestore write:** Best-effort set() on games/{roomId}, errors logged.
- **Observation:** callbackFlow combining a Firestore snapshot listener with the in-memory flow.

All Firestore calls are wrapped in try/catch; firestore is nullable and silently skipped if not initialized.

X. Screen Layouts

A. Login Screen

B. Lobby Screen

C. Waiting Room Screen

D. Game Screen

1) Screen Zones:



Figure 6. Login screen layout.

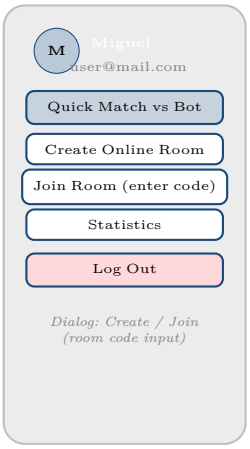


Figure 7. Lobby screen layout.

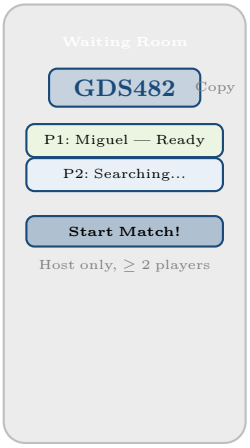


Figure 8. Waiting room screen layout.

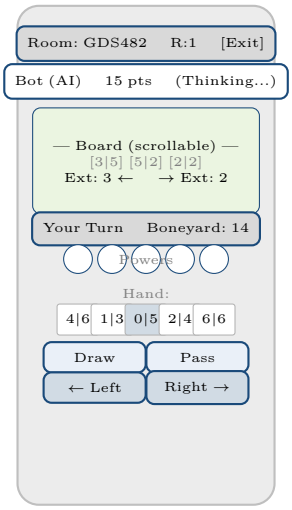


Figure 9. Game screen layout. From top: room info bar, opponent header, board with open ends, turn status and boneyard count, power icons, player hand (center tile selected), draw/pass and direction controls.

1) **Top:** Room ID / round / exit button. Opponent avatar, name, thinking indicator, score.
2) **Center:** `GameBoardView` (horizontal scroll). `RandomEventBanner` if active.
3) **Powers:** Row of circular `IconButtons` with charge badges.
4) **Bottom:** Turn label, boneyard count, player hand (`LazyRow` of `DominoTileView`), Draw/Pass and direction buttons.

On `FINISHED`: bottom replaced by "Match Ended!" card with return button; `ConfettiOverlay` shown; victory sound played.

2) Overlaid Dialogs:

- **CoinTossDialog:** When `status == COIN_TOSS`; host calls `finishCoinToss()`.
- **PowerConfirmDialog:** When `pendingPowerDialog != null`; shows power details; SWAP shows tile hand.

E. Statistics Screen

XI. Complete Game Flow

Figure 11 illustrates a complete match from start to termination.

XII. Firebase Data Schema

The `users/{uid}` collection stores `uid`, `email`, `displayName`, and `lastLogin`. The in-memory store ensures instant same-process propagation; cross-device sync relies on Firestore snapshots.

XIII. Conclusion

GODSMINO demonstrates a complete Android game implementation following best practices. Key technical contributions:

- Strict MVVM with reactive state management via `StateFlow`.

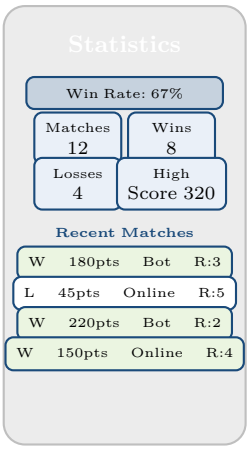


Figure 10. Statistics screen layout.

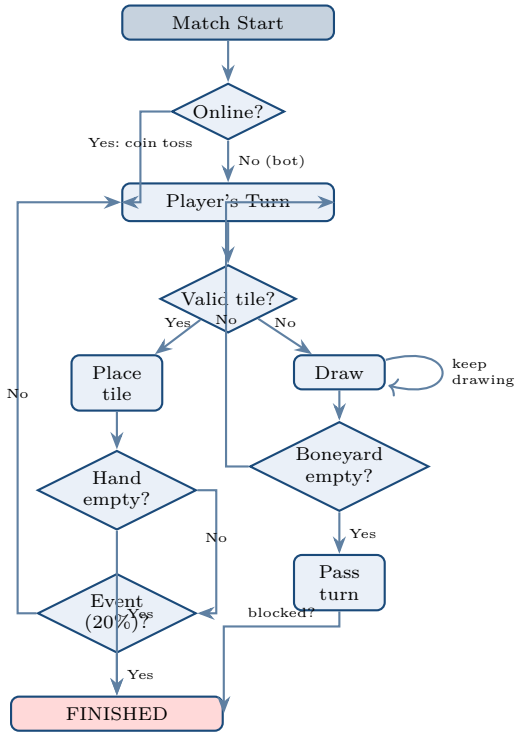


Figure 11. Complete game flow including coin toss, turn cycle, random events, and termination conditions.

Table XII
FIRESTORE DOCUMENT STRUCTURE: `GAMES/{ROOMID}`

Field	Type / Value
`roomId`	String
`board`	Array of Tile
`players`	Array of Player
`currentTurnPlayerId`	String
`boneyard`	Array of Tile
`status`	GameStatus enum (String)
`activeEvent`	RandomEvent object
`roundNumber`	Int

- A self-contained game engine in the repository layer, fully decoupled from the UI.
- Hybrid Firestore/in-memory synchronization with graceful offline degradation.
- Five special powers with limited charges, validated on every use.
- A three-condition scoring system (domino, blocked, forfeit) with a power-usage tiebreaker.
- Runtime PCM audio synthesis without media file assets.

Known limitations: strictly 2-player (design specified 2–4); Firestore writes are fire-and-forget without conflict resolution; the in-memory store is per-process, so cross-device sync depends entirely on Firestore.

REFERENCES

[1] Google, "Jetpack Compose," https://developer.android.com/jetpack/compose, 2024.
[2] Google, "App Architecture Guide," https://developer.android.com/topic/architecture, 2024.
[3] Google, "Firebase Firestore," https://firebase.google.com/docs/firestore, 2024.
[4] JetBrains, "Kotlin Docs," https://kotlinlang.org/docs/home.html, 2024.
[5] JetBrains, "Kotlin Coroutines," https://kotlinlang.org/docs/coroutines-overview.html, 2024.